

PIPO: Privacy-Preserving Convolutional Neural Network Inference with Plaintext Operations

Tian Zhou*, Lixin Gao*

* Department of Electrical and Computer Engineering, University of Massachusetts Amherst, USA

tzhou@umass.edu, lgao@ecs.umass.edu

Abstract—Neural network models have become essential building blocks for cloud-based services. However, these services require users to upload their data to the cloud and thereby potentially reveal private data about users. Privacy-preserving neural network inference has been proposed to protect users' data as well as the server's neural network models. These methods rely heavily on cryptographic operations which are compute-intensive. In this paper, we propose PIPO, a privacy-preserving framework for convolution neural network (CNN) inference. The key idea of PIPO is to accelerate the operations in neural networks by avoiding expensive cryptographic operations as much as possible. In particular, the client preprocesses the inference by performing linear operations, such as convolution, on a secret share of the input through homomorphic encryption. The user only needs to provide the rest of the secret shares of the input to the server to perform convolution on plaintext during the online inference. As a result, PIPO protects users' data while performing the inference with plaintext operations. To prevent model parameters from being revealed to the client directly, the server performs two reversible operations before sending intermediate results to the client: randomly scaling each entry with different scale factors and shuffling them. We evaluate PIPO on well-known CNN architectures and datasets and show that PIPO reduces the inference latency and communication volume by up to 78x and 26x respectively compared with Delphi [1]. The source code is available at GitHub.

I. INTRODUCTION

Deep learning has been widely adopted in myriad applications, such as image classification [2], object detection [3], and natural language processing [4]. Powerful deep learning models are usually built with a large amount of data and model parameters, and inference using these models requires a huge amount of compute resources [4], [5]. As a result, neural network inference is performed on the cloud. That is, a user needs to upload its data, such as images, videos, and text, to the cloud service provider. This raises privacy issues since the cloud service provider may leak or use the user data for other purposes. At the same time, malicious users may steal the machine learning (ML) model from the cloud. Note that such ML models are usually valuable intellectual properties since training a good model usually costs a huge amount of time and compute resources. Therefore, enabling neural network inference in a privacy-preserving manner is critical for cloud-based machine learning services.

One approach to performing privacy-preserving neural network inference is to use the homomorphic encryption (HE) technique. It allows performing arithmetic operations (addition and multiplication) on encrypted data [6]–[8]. Users encrypt

their inputs with homomorphic encryption [9]–[11], and let the server perform the operations on the encrypted data. At the end of the inference, the server sends the encrypted result to the user, and the user decrypts it to get the inference result. However, computations in the ciphertext space are usually hundreds of thousand times slower than plaintext computations. In addition, popular HE schemes, such as CKKS [7], do not directly support non-linear operations, such as the ReLU activation function and the max pooling operation. They usually approximate the non-linear operation with a polynomial function [9], [12], which is costly in HE.

Recently, researchers propose to perform linear operations by offloading the expensive HE workload to an offline phase and performing only plaintext operations during the online inference [1], [13]–[15]. For the non-linear operations, such as ReLU and max pooling, researchers [1], [10], [11], [15]–[18] move them to the client and use garbled circuits (GC) [19] or function secret sharing (FSS) [20], [21] to avoid revealing model parameters to the client. However, GC and FSS implementations are computationally expensive. For example, current ReLU implementations under GC are several orders of magnitude slower than plaintext ReLU [1], [13].

In this paper, we present PIPO, a fast privacy-preserving inference system for convolutional neural networks. PIPO conducts the inference in plaintext while ensuring that the user data is not revealed to the cloud server. PIPO protects the user input by sharing only a secret share of the input to the cloud server and convolution operation of the rest of the secret share is performed offline using HE. Therefore, the convolution operations are performed in plaintext during the online inference phase. It greatly reduces the inference latency and communication volume.

PIPO performs non-linear operations, such as ReLU and max pooling, on the client side. In order to prevent model parameters from being revealed to the client directly, the server performs two reversible operations: multiplying each entry of the convolution results with a randomly generated scale factor and shuffling the results. Before sending a convolution result back to the client, the server protects the value of each entry by multiplying it with a randomly generated positive scale factor, and protects the locations of the entries by randomly shuffling them. As a result, the client can perform the element-wise non-linear operations, represented by ReLU, and regional non-linear operations, represented by pooling, without knowing the actual values of the entries. Since scaling and shuffling are

reversible for common non-linear operations, the server can recover designated results from the client's output.

We show PIPO ensures that the client's input data is secure against a semi-honest server. Further, we analyze the resources required to steal the model parameters. We compare it with the prediction API attack, which is an attack that the client can perform on any inference service since both inputs and inference results are known to the client. Our analytic results show that the resources required to steal the model parameters in PIPO are at least half of the resources required by the state-of-the-art prediction API attack.

We summarize our contributions as follows:

- We design PIPO, a privacy-preserving protocol for neural network inference. The key idea is to forgo the expensive cryptographic protocols. The proposed protocol greatly speeds up the inference. Further, PIPO is able to perform the inference with the exact neural network model without approximation, thereby preserving the accuracy achieved by the neural network model.
- We analyze the security of the proposed protocol. We first prove that the user's data is protected against honest-but-curious (semi-honest) servers. Further, we analyze the number of queries a semi-honest client needs to extract the model parameters from PIPO. We also build a prediction API attack against CNNs based on the state-of-the-art model extraction methods [22], [23] and compare the resources needed for the model stealing attack on PIPO. Our results show that the number of queries required for the PIPO attack is within the same order of magnitude as the state-of-the-art prediction API attack.
- We implement PIPO and evaluate its performance on well-known neural network architectures and real-world datasets. Our results show that the inference of PIPO is 21x and 78x faster than Delphi on the MiniONN and ResNet-32 respectively. The communication volume of PIPO is up to 26x less than Delphi. In addition, PIPO improves the accuracy on MiniONN and ResNet-32 by 7.6% and 10.6% respectively over Delphi, since PIPO supports ReLU and max pooling without using approximation.

The rest of this paper is organized as follows. Section II describes the cryptographic primitives and key mathematical tools used in the system. Section IV presents our protocol for secure neural network inference. Section V analyzes the security of the proposed protocol. Section VI evaluates the performance of the proposed system. Section VII discusses related work. Finally, Section VIII concludes the paper.

II. BACKGROUND

In this section, we briefly review cryptographic primitives used in this paper: linear homomorphic encryption and additive secret sharing.

A. Linear Homomorphic Encryption

A linear homomorphic encryption scheme is a public key encryption scheme that supports linear operations on

ciphertexts. Specifically, a linear homomorphic encryption, $HE = (KeyGen; Enc; Dec; Eval)$, supports the following operations.

- $HE.KeyGen() \rightarrow (pk, sk)$: Generate a public key pk and a secret key sk .
- $HE.Enc(pk, m) \rightarrow c$: Encrypt a message m to a ciphertext c .
- $HE.Dec(sk, c) \rightarrow m$: Decrypt a ciphertext c to a message m .
- $HE.Eval(pk, c, f) \rightarrow c'$: Evaluate a linear function f on a ciphertext c .

A linear homomorphic encryption scheme satisfies the following linear properties for any messages m_1 and m_2 and any constant C :

$$Enc(pk, m_1) \oplus Enc(pk, m_2) = Enc(pk, m_1 + m_2) \quad (1)$$

$$Enc(pk, m_1) \otimes C = C \otimes Enc(pk, m_1) = Enc(pk, m_1 \cdot C) \quad (2)$$

where $\oplus$ and $\otimes$ denote the addition and multiplication in the ciphertext space.

B. Additive Secret Shares

In this paper, we adopt the additive secret shares for secure two-party computation. It allows the two parties to share a secret x such that none of them can reconstruct it using its own share. Mathematically, given a ring $\mathbb{R}$, a 2-of-2 additive secret shares of x is a pair $([x]_1, [x]_2) = (x - r, r) \in \mathbb{R}^2$ (so that $x = [x]_1 + [x]_2$) where $r \in \mathbb{R}$ is a random element. Additive secret sharing scheme is perfectly hiding, *i.e.*, given a share $[x]_1$ or $[x]_2$, the value x is perfectly hidden.

C. Convolutional Neural Network

Convolutional Neural Network (CNN) is a class of deep neural networks that are based on the convolution operation. It is widely used in computer vision tasks, such as image classification, object detection, and semantic segmentation.

The convolution used in CNN is 2D convolution over multiple channels. The input data is separated into many small 2D blocks and performed a linear operation on each block with the same parameter, known as a convolution kernel. Each kernel is characterized by the width, height, and depth of its receptive field. The kernel depth equals to the number of channels of the input feature map. For each kernel $W \in \mathbb{R}^{c \times h \times w}$ of height h , width w and depth c , convolution operation over an input $X \in \mathbb{R}^{c \times h' \times w'}$ with c channels and $h' \geq h, w' \geq w$, can be represented as follows:

$$Y = X * W = [y_{i,j}]_{(h'-h+1) \times (w'-w+1)} \quad (3)$$

where each output entry $y_{i,j}$ is:

$$y_{i,j} = \sum_{i_c=1}^c \sum_{i_h=1}^h \sum_{i_w=1}^w x_{i_c, i+i_h, j+i_w} \cdot w_{i_c, i_h, i_w} \quad (4)$$

III. THREAT MODEL AND PRIVACY GOALS

A. Threat Model

Our threat model assumes that the client and server are semi-honest. That means the client and server follow the protocol but try to learn the other party's private data. The server tries to learn the client's input data using its view, which consists of the messages received from the client during the protocol interaction.

Similarly, the client tries to learn the server's neural network model based on its view, *i.e.*, the messages received from the server during the protocol interaction. We assume that the server does not hide information about the architecture of the neural network, such as the number of layers, the number of convolution channels, and the dimensions of each layer in the network. The client, however, aims to discover the neural network's model parameters.

B. Privacy Goals

For the privacy of a client, we aim to design a protocol where the server performs neural network inference without knowing the client's input. In addition, after performing the inference, the server should not know anything about the inference output.

As for the privacy of a server, like most prior systems for privacy-preserving prediction [1], [10], [16], we do not aim to hide information about the prediction result from the client. A curious users can always collect input-output pairs using the prediction API of the server and learn the model parameters with them. This is referred to as the prediction API attack. Therefore, for the privacy of a server, we aim to make the client take as many prediction queries as possible to learn the model parameters.

IV. PIPO PROTOCOL

In this section, we introduce our protocol for privacy-preserving convolutional neural network inference. We first give an overview and the key ideas of our protocol in Section IV-A. Then, we introduce our protocol for common layers.

A. Overview

We propose PIPO, a fast privacy-preserving inference system for convolutional neural networks. The key idea of PIPO is to avoid expensive and time-consuming cryptographic operations. To this end, PIPO uses a preprocessing step to precompute the convolution operation on a secret share. As a result, convolution operations during the online phase only need to be performed on the rest of the input, which appears to be random to the cloud server. The preprocessing step uses homomorphic encryption to protect the secret share from being revealed to the server. The CNN inference performed online uses only plaintext operations.

However, nonlinear CNN operations such as ReLU can not be performed at the server since the server does not have the complete convolution results. It is necessary to perform ReLU on the client side. In order not to reveal the convolution results to the client, the server uses randomly generated scale factors

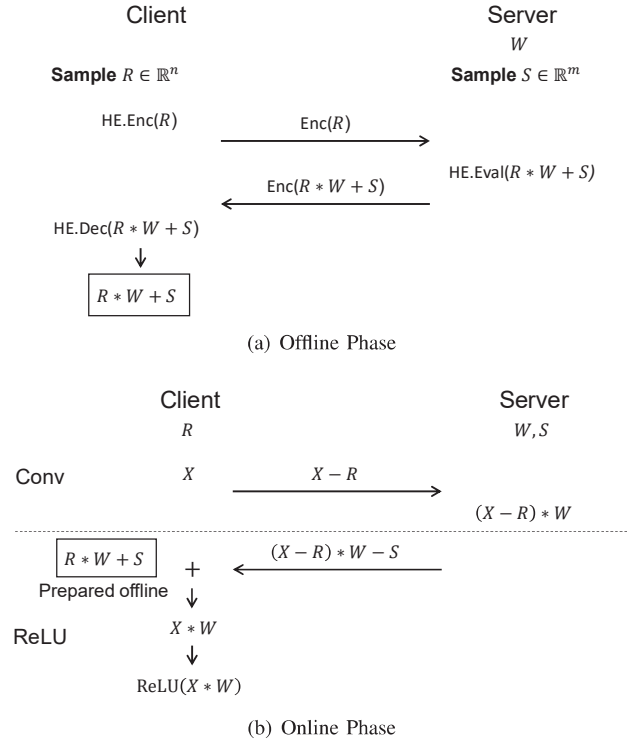


Fig. 1. Basic Protocol for Convolution and ReLU

to multiply the corresponding entries of the convolution results and shuffle these entries before sending them to the client for the ReLU operation. The server side stores both the scale factors and shuffle mappings so that the reverse operations can be performed at the server when the ReLU results are sent back from the client. These scale factors and shuffle operations are the key to preventing the model parameters from being revealed to the client directly.

Next, we show how PIPO works. For input data X (a vector representing pixel values of a frame), the client generates additive secret shares $[X]_C = R$ and $[X]_S = X - R$ using an offset R (a vector of the same dimension as X and each entry is a randomly generated value). Since the client sends the server only $X - R$, X is hidden from the server. The server performs linear operations, such as convolution, fully-connected, and normalization, on both R and $X - R$. The convolution operation for X using model parameter W is represented as $X * W$. The convolution of R with W is precomputed offline by the server using homomorphic encryption. We can reconstruct the result $X * W$ by adding the result of $(X - R) * W$ and the precomputed $R * W$. Fig. 1 shows both the online and offline protocols for convolution. Note that the server performs linear operations, and sends the result to the client with a random offset S in order not to reveal the convolution result of $(X - R) * W$ to the client. The client holds the precomputed result, $R * W - S$, and adds it to the result from the server.

The above basic protocol is vulnerable to the client's model stealing attack for W using X and $X * W$. We propose to use

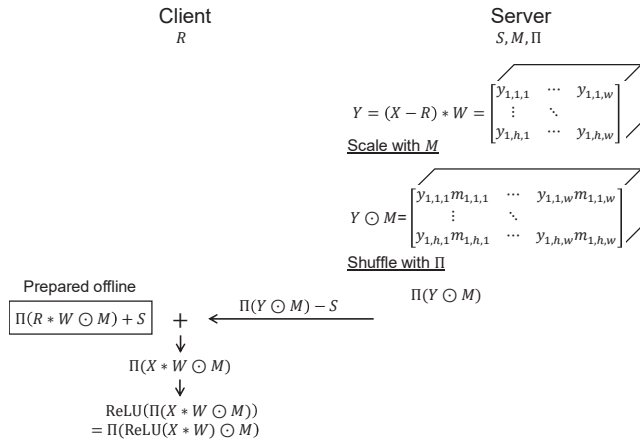


Fig. 2. ReLU Operation with Scale Factors and Shuffle

scale factors and shuffle to protect the model parameters from the client. In order to prevent the client from knowing the exact value of $X * W$, we exploit the property that $\text{ReLU}(y \cdot m) = \text{ReLU}(y) \cdot m$ for any y and any positive factor m . This property holds for other non-linear operations such as max pooling as well. That is, $\text{ReLU}(Y \odot M) = \text{ReLU}(Y) \odot M$ for any input $Y \in \mathbb{R}^n$ and any positive scale factor tensor $M \in \mathbb{R}^{+n}$, where $\odot$ is the element-wise multiplication operator. In practice, the server generates M randomly each of which is within the range between 0 and D (preventing overflow in the discrete field), and stores them so that it can scale back the result after receiving its corresponding non-linear operation result. Note that we use a different scale factor for each entry of Y . Further, to prevent the client from knowing the exact entry that it performs ReLU on, the server randomly shuffles entries of $Y \odot M$ before sending them to the client. The shuffle mapping Π is stored at the server so that we can shuffle the entries back using Π^{-1} . The scale factors and shuffle increase the difficulty of stealing the model parameters, which we will discuss in Section V-B. Note that we also perform scale factor multiplication and shuffle in the offline phase during preprocessing on the server side. As a result, the client side has the corresponding part from the preprocessing. Fig. 2 shows how scale factor multiplications and shuffle are performed at the server side and ReLU is performed at the client side.

B. First Convolution+ReLU Layer

Here, we introduce our protocol for computing the first convolution layer together with its successive ReLU activation. At the beginning of the inference, the client has the input data X_1 and the server has the convolution kernel W_1 . After the computation, the client will have a scale-and-shuffle version of $\text{ReLU}(X_1 * W_1)$.

The protocol is separated into two phases, the offline preprocessing phase and the online inference phase. In the offline preprocessing phase, we prepare the secret shares, scale factors, shuffle mappings, and the precomputed intermediate result as introduced above. In the online inference phase, with

the help of the prepared data, the client and server conduct the inference using plaintext operations.

1) *Online Inference Phase:* To keep the linear weights W_1 private against the client, we use the scale factor. Since the client has X_1 , sending $(X_1 - R_1) * W_1$ to the client reveals the linear weights W_1 . So the server protects $(X_1 - R_1) * W_1$ with the random scale factor by multiplying a random scaling factor $M_i \in \mathbb{R}^{n_1}$, where n_1 is the number of elements in the first layer's output. And the server holds the scale factor tensor M_i . The client reconstructs the linear result of $X_1 * W_1 \odot M_1$ by adding $(X_1 - R_1) * W_1 \odot M_1$ and $R_1 * W_1 \odot M_1 - S_1$. Fig. 3(b) shows the protocol for the online phase. We describe it as follows.

- 1) The client generates the additive secret share of input data X_1 by adding the prepared secret share R_1 onto X_1 and sends the server's share $X_1 - R_1$ to the server.
- 2) The server performs the linear operation over the received secret share and gets $Y' = (X_1 - R_1) * W_1$.
- 3) The server scales and shuffles Y' with prepared the random scale factor tensor M_1 and mapping Π_1 respectively, and gets $\Pi_1(Y' \odot M_1)$.
- 4) The server generates the additive secret share of the above results with a prepared secret share S_1 and sends $\Pi_1(Y' \odot M_1) - S_1$ to the client.
- 5) The client uses the prepared $\Pi_1((R_1 * W_1) \odot M_1) + S_1$ to recover $\Pi_1((X_1 * W_1) \odot M_1)$.
- 6) The client performs the ReLU activation over $\Pi_1(X_1 * W_1 \odot M_1)$ and gets $\text{ReLU}(\Pi_1((X_1 * W_1) \odot M_1)) = \Pi_1(\text{ReLU}(X_1 * W_1) \odot M_1) = \Pi_1(X_2 \odot M_1)$.

2) *Offline Preprocessing Phase:* The offline preprocessing phase prepares the additive shares used in the online inference phase. Since it involves both the private data, R_1 , of the client and the private data, M_1 , of the server, we apply the homomorphic encryption for the offline preprocessing phase as shown in Fig. 3(a). We describe the protocol as follows.

- 1) The client runs $HE.KeyGen$ to obtain a public key pk and a secret key sk .
- 2) The client generates a random error vector $R_1 \in \mathbb{R}^{n_0}$, where n_0 is the dimension of the input data X .
- 3) The client encrypts R_1 with $HE.Enc(pk; R_1)$ and sends it to the server.
- 4) The server generates a random offset $S_1 \in \mathbb{R}^{n_1}$ and a random scale factor $M_1 \in \mathbb{R}^{n_1}$, where n_1 is the dimension of the output of the input layer.
- 5) The server computes $Y = HE.Enc(pk; \Pi_1((R_1 * W_1) \odot M_1) + S_1)$ and sends the resulting ciphertext to the client.
- 6) The client computes $HE.Dec(sk; Y)$ and stores the result.

C. Later Convolution+ReLU Layers

At the beginning of the computation for layer $i > 1$, the client holds $\Pi_{i-1}(X_i \odot M_{i-1})$, while the first layer holds its X_i directly. So to conduct the convolution operation correctly, the protocol first recovers X_i and then performs the convolution and ReLU computation.

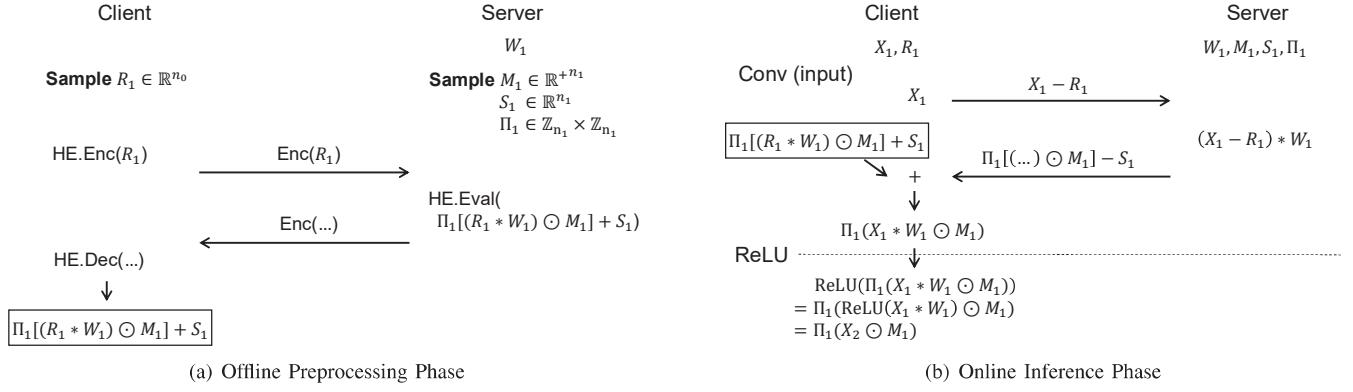


Fig. 3. Protocol for the First Convolution+ReLU Layer

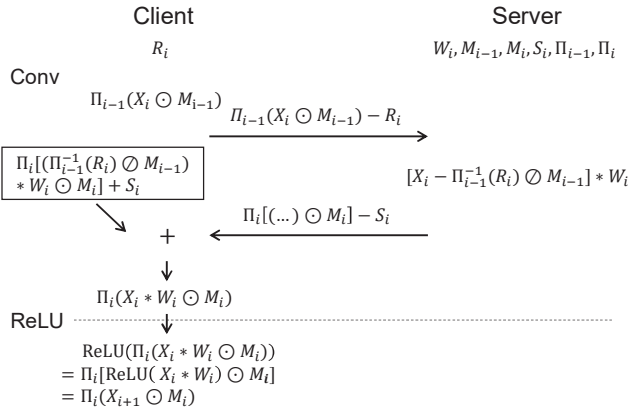


Fig. 4. Protocol for Middle Convolution+ReLU Layer (Online Phase)

In the online phase, the client sends the additive share of $\Pi_{i-1}(X_i \odot M_{i-1})$ with R_i to the server. The server recovers X_i from the received $\Pi_{i-1}(X_i \odot M_{i-1}) - R_i$ by applying the last layer's scale factors M_{i-1} and shuffle mapping Π_{i-1} reversely, as follows where $\odot$ is the element-wise division operator and Π_{i-1}^{-1} is the reverse of Π_{i-1} .

$$\begin{aligned} & \Pi_{i-1}^{-1}((\Pi_{i-1}(X_i \odot M_{i-1}) - R_i) \odot M_{i-1}) \\ &= X_i - \Pi_{i-1}^{-1}(R_i) \odot M_{i-1} \end{aligned} \quad (5)$$

Denoting $R'_i = \Pi_{i-1}^{-1}(R_i) \odot M_{i-1}$, the server can then perform convolution and ReLU as the first layer for $X_i - R'_i$. Correspondingly, they need to prepare $\Pi_i(R'_i * W_i \odot M_i) + S_i$ in the offline phase. We illustrate the online phase of the protocol for middle layers in Fig. 4.

Since the last layer returns the final result, which should be interpretable to the client, the server does not need to scale and shuffle the convolution result when sending it back. So, it just recovers the convolution input and performs the convolution operation.

D. Max Pooling Layer

The max function also has the positive homogeneity i.e., $\max(mx) = m \max(x)$ for any $x \in \mathbb{R}^n$ and positive scale

factor $m \in \mathbb{R}^+$. Since the pooling stride usually equals to or greater than the pooling size, we can perform the max pooling correctly by using the same scale factor for entries in the same pooling region. Similarly, entries of the same pooling region should be shuffled to adjacent locations.

Fig. 5 shows the protocol for the max pooling layer. Since the shape of the data is important for pooling, instead of simplifying the dimension with n_i , we use $c_i \times h_i \times w_i$ here ($c_i h_i w_i = n_i$). Different from ReLU, the protocol for max pooling replaces the fully random scale factors M_{i-1} and shuffle mapping Π_{i-1} . First, the client sends current data to the server. The server reorganizes the input data with a new shuffle Π'_i that puts entries of the same pooling region close to each other and scales the data with M'_i that gives the same scale factors for entries in the same pooling region. Then, the server sends the data back to the client for pooling.

In the offline phase, the client first randomly generates scale factors $M_i \in \mathbb{R}^{+c_{i+1} \times h_{i+1} \times w_{i+1}}$ based on the shape of the output of the max pooling layer. Then, the client computes scaling factor $M'_i \in \mathbb{R}^{+c_i \times h_i \times w_i}$ for the input of the max pooling operation by expanding M_i to size $c_{i+1} \times h_{i+1} \times w_{i+1}$ and duplicating the scale factors for entries in the same pooling region. Assuming the pooling stride is (h, w) and the stride equals or larger than the pooling kernel size, we have $M'_i = 1^{1 \times h \times w} \otimes M_i$ where $\otimes$ denotes Kronecker product. Similarly, we construct Π'_i to keep entries of the same pooling region together and only shuffle across pooling regions. So, we have M'_i and Π'_i for the pooling's input, and M_i and Π_i for the pooling's output. Using M'_i and Π'_i , the server and client prepare the offline data for reorganizing the last layer's output.

V. PRIVACY ANALYSIS

In this section, we analyze the security of PIPO for both the user's data and the model parameters.

A. Privacy of User's Data

Theorem 1. Assuming the existence of a secure homomorphic encryption scheme, PIPO is secure against a corrupted, semi-honest server.

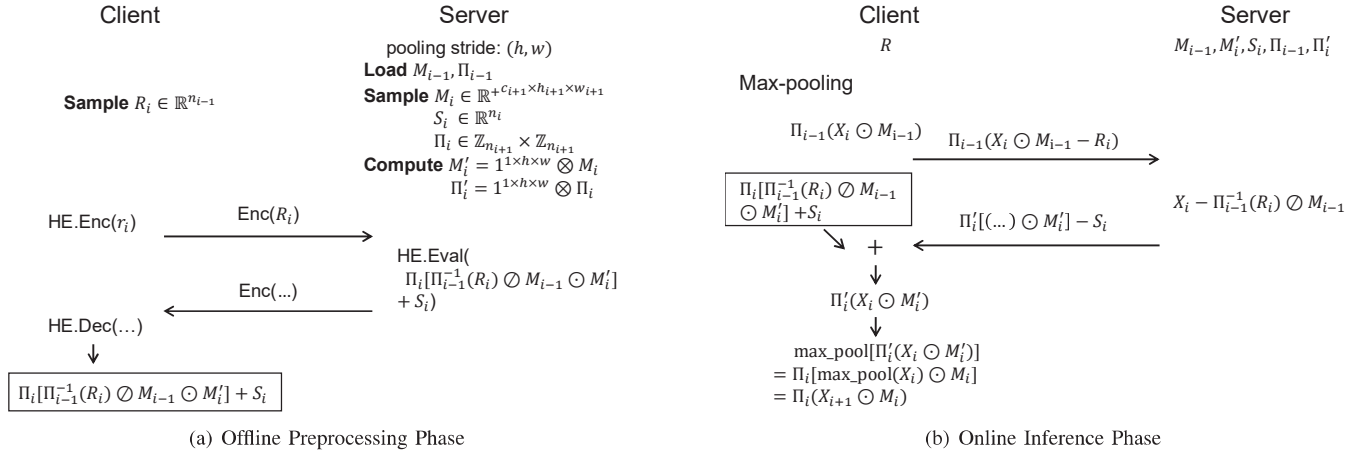


Fig. 5. Protocol for Max Pooling Layer i

Proof. The theorem about PIPO's security means that a semi-honest server cannot learn the input data X of the client. Formally, it can be proved by showing the existence of an efficient simulator Sim_C of the client such that it is computationally indistinguishable with a real client from the view of a sever, i.e., $View_S \approx_c Sim_C$, where $View_S$ is the view of the server when cooperating with a client.

We prove it by constructing a simulator Sim_C of the client that does not use the actual input value X , and showing that the real-world distribution is computationally indistinguishable from the simulated distribution via a hybrid argument.

Simulator Construction: Provided with the server's W_i for all layers, Sim_C proceeds as follows to cooperate with the corrupted server:

1. Sim_C chooses a uniform random tape for the server.
2. In the offline phase, Sim_C first choose a pk for the homomorphic encryption scheme. For each layer i , instead of sending the $HE.Enc(pk; R_i)$ to the server, Sim_C encrypts junk data such as $\mathbf{0}$ and sends the ciphertext $HE.Enc(pk; \mathbf{0})$ to the server. In return, it receives a homomorphically evaluated ciphertext from the server.
3. In the online phase, for each layer i , Sim_C sends a uniformly generated random data $R'_i \in \mathbb{R}^{c_i \times h_i \times w_i}$ to the server. Then, it receives the corresponding computation result from the server.

Hybrid Argument: We show that the real-world distribution is computationally indistinguishable from the simulated distribution via a hybrid argument. In the final simulated distribution, the simulator Sim_C does not use the client's input X , and so a corrupted server learns nothing in the real world.

- 1) Hyb_0 : This corresponds to the real-world distribution where the client uses its actual input X .
- 2) Hyb_1 : In this hybrid, we replace the offline phase with the simulator's offline phase. Based on the security of the HE scheme, the ciphertext of the junk data is indistinguishable from the ciphertext of the real data R_i . So Hyb_1 is computationally indistinguishable from Hyb_0 .

- 3) Hyb_2 : In this hybrid, we replace the online phase with the simulated online phase. The server receives the random data R'_i from the simulator. In the real world, the server receives one additive share of $\Pi_{i-1}^{-1}(X_i \odot M_{i-1})$, which is random in $\mathbb{R}^{c_i \times h_i \times w_i}$. So R'_i is indistinguishable from $\Pi_{i-1}^{-1}(X_i \odot M_{i-1}) - R_i$ (one share) from the view of the server. So, Hyb_2 is distributed identically to Hyb_1 . Note that, the real input data X is not used in this hybrid.

Finally, note that Hyb_2 is identically distributed to the simulator's output, we have the proof. $\square$

B. Privacy of Model Parameters

Before, getting to the detailed analysis, we think it is important to realize that model parameters cannot be perfectly protected against the client in any privacy-preserving inference protocol, as long as the inference result is given back to the client. Mathematically, if and only if the outputs are computationally indistinguishable from random values, the client can never learn the model parameter from the protocol. However, a meaningful CNN inference result is designed to encode information about the input and should not be random. That leaves space for the client to learn the model parameters. For example, the client can always learn the model parameters by collecting input-output pairs and training a surrogate model by itself.

Therefore, our privacy argument about model parameters is about how many resources the client takes to learn them (or an equivalent version). More specifically, we focus on the number of required queries. We first compose an adaptive attack against PIPO based on the state-of-the-art model stealing techniques. It makes use of both the input-output pairs and intermediate results. Then, we introduce a prediction API attack against CNNs, which only uses input-output pairs, based on state-of-the-art techniques. By comparing the number of queries required by the two attacks on some common CNN architectures, we show that the resources required to steal model parameters from PIPO are at least half of the state-of-the-art prediction API attack against CNN.

We only provide the main idea of the two attacks and their comparison here. More details about the attack methods and analysis of the required number of queries are provided in the appendix. (Omitted here due to the page limit for papers under review.)

1) *Resources Required by Adaptive Attack against PIPO:*

The key idea of the attack is to learn an equivalent version of model parameters. More precisely, the extracted model parameters of each layer i can be α_i times of the original one, or the order of kernels may be different from the original one, but the final output of the extracted model is the same as the original model.

We introduce the method of learning the first layer here. The idea can be easily extended to deeper layers. For the first layer, the client knows the input X and output $\Pi_1(X * W_1 \odot M_1)$.

a) *Get rid of scale factors:* Suppose Π_1 is known, the client can launch multiple similar queries and compare their outputs at the same output location, where the scale factor is fixed. To learn a convolution kernel $w \in W_1$ ($w \in \mathbb{R}^{c \times k \times k}$), we can construct the following queries that only activate one entry of w each time and align the output at output location (p, q) .

$$\begin{aligned} [f_1(x + \mathbf{I}_{1,p,q}) - f_1(x)]_{p,q} &= w_{1,1,1} \cdot m_{p,q} \\ [f_1(x + \mathbf{I}_{1,p,q+1}) - f_1(x)]_{p,q} &= w_{1,1,2} \cdot m_{p,q} \\ &\dots \end{aligned} \quad (6)$$

where x is a valid input, $\mathbf{I}_{i,j,k}$ is the one-hot tensor with a one at location (i, j, k) , and $m \in M_1$ represents the scale factors corresponding to this convolution kernel. We can get a scaled version of w as $\hat{w}$:

$$\hat{w} = \frac{w_{i,j,k}}{w_{1,1,1}} = \frac{[f_1(x + \mathbf{I}_{i,p+j,q+k}) - f_1(x)]_{p,q}}{[f_1(x + \mathbf{I}_{1,p,q}) - f_1(x)]_{p,q}} \quad (7)$$

Since all convolution kernels in the same layer take the same input and the client can get all kernels' outputs with one query, we just need $N_w^{(1)} = c_0 k^2$ queries to learn all $c_1 c_0 k^2$ weights.

b) *Get rid of permutation:* The permutation is performed on all three dimensions of the data (channel, height, and width).

For the permutation on the height and width dimensions, we can learn it using the fact that changing one input entry of a convolution affects all output entries whose receptive fields cover the changed input entry, i.e., $c_i k^2$ entries. And given the coordinates of the input entry, the coordinates of the affected output entries can be easily computed. For the shuffled output, we can identify affected outputs by comparing the outputs before and after changing the input. So by crafting two sets of queries whose affected output regions just overlap by one entry, we can map the shuffled coordinate of the overlapped entry with its original coordinate. After that, we can craft further queries to learn the mappings around the solved one by moving the affected outputs by one each time around the known entry.

Now, we consider the channel dimension in the output. Each height-weight location is a vector of length c_1 . The c_1 values are shuffled. And on different h-w locations, the

shuffle mappings are different. If we just want to know the model parameters of the first layer, we can ignore the shuffle and extract the kernels in arbitrary order. To learn the model parameters for the next layer, we need to identify the entries that relate to the same kernel on each h-w location. Due to the scale factors, we cannot directly align the entries. However, all the scale factors are positive, which means that the sign of $X * w$ is maintained. So we can align the output channels by adjusting the input X to make there is only one output channel's sign flipped. We use the critical point idea from [23] to search for the X that flips the sign of only one output channel. In short, we search the input space with some probe lines that linearly change X . In this case, $X * w$ is linear with the change of X . So we use a binary search to search for the flipping point for each output channel.

We need $c_1 k^2$ shuffle mappings. On the height-width dimensions, there are k^2 locations, we resolve each of them with 4 queries. At each height-width location, there are c_1 channels to be aligned. We can align each of the output channels with $\log_2(\lambda)$ queries, where λ is the resolution of the attack. Therefore, we need $N_s^{(1)} = 4r_1^2 + c_1 \log_2 \lambda$ queries to learn the shuffle for the first layer, where $r_i = 1 + (k-1)(L-i)$ is the radius of the receptive field of layer i .

c) *Number of queries:* For an L -deep CNN with $k \times k$ convolution kernels, the attack method requires at least the following number of queries.

$$\begin{aligned} &\sum_{i=1}^L (c_{i-1} k^2 + c_i r_i^2 + 4r_i^2 + c_i \log_2 \lambda) \\ &\approx \sum_{i=1}^L (2(L-i)^2 c_{i-1} k^2 + c_i \log_2 \lambda) \end{aligned} \quad (8)$$

2) *Resources Required for Prediction API Attack:* The prediction API attack is a type of model-stealing attack that only utilizes the inference input and output. Currently, the state-of-the-art prediction API attack methods are based on the reverse-engineering techniques over ReLU-based networks [22]–[24]. Despite existing works focusing on fully connected networks, we find that the idea can be easily extended to CNNs and achieves better performance utilizing the properties of CNNs.

The key idea of this method is to use a relationship between the partial gradient of the network and the model parameters from [23]. For a fully connected network f , its gradient $\nabla f = \partial f / \partial X = W_L I_{L-1} W_{L-1} \cdots W_1$, where W_k is the model parameter of the k -th layer and I_k is a binary diagonal matrix that represents the gradient of the ReLU operation. Each entry of I_k is either 1 or 0 and only switches when its input flips its sign. It leads to the fact that I_k is stable within many small regions in the input space i.e., signs of all entries of I_k for all $1 \leq k \leq L$ are not changed. Therefore, if we find a specific input x^* in such a region Ω and it activates a ReLU operation of the first layer, we have:

$$\partial f / \partial X_k = W_L I_{L-1}(x^*) W_{L-1} \cdots W_2 \mathbf{1}_{w_{1,k}} \quad (9)$$

Based on it, we can get rid of everything other than W_1 and extract an equivalent version of W_1 as follows (suppose $I_k(x^*) \neq \mathbf{0}$ for all $k > 1$):

$$\hat{w}_{1,k} = \frac{w_{1,k}}{w_{1,1}} = \frac{\frac{\partial f}{\partial X_k}|_{X=x^*+\epsilon_k} - \frac{\partial f}{\partial X_k}|_{X=x^*-\epsilon_k}}{\frac{\partial f}{\partial X_1}|_{X=x^*+\epsilon_1} - \frac{\partial f}{\partial X_1}|_{X=x^*-\epsilon_1}} \quad (10)$$

In CNNs, a convolution only operates on input entries within its receptive field. So, different from fully connected networks, in such a region Ω that can be used to extract the model parameters of the first layer, the gradient ∇f only has non-zero entries in its receptive field. Since the receptive fields of different layers are in different sizes, we can easily distinguish gradient entries of different layers. In addition, since each convolution kernel repeats multiple times, we can extract the same model parameter multiple times at different input locations. But actually, we only need to extract it. The majority of the attack workload is to find the special regions.

We utilize the above properties of CNNs to reduce the number of queries. First, we can quickly check the layer a critical point belongs to. Second, we can reduce the number of queries for probing using the convolution property.

For a convolution kernel in a CNN, only the X entries within its receptive field lead to non-zero partial gradients. For example, there is only one 3-by-3 non-zero block in the gradient matrix $\nabla f|_{x_1}$ of a critical point x_1 in the first layer. And, all other entries in the gradient matrix are zero. For a critical point x_2 of the second layer, there must be a 5-by-5 non-zero block (convolution of two 3-by-3 kernels) in $\nabla f|_{x_2}$. Therefore, we can easily select critical points of the first layer without additional queries.

In CNN, the critical points belonging to later layers are stored and reused to reduce the workload of probing later layers. After the model parameters of the first layer are extracted, we can compose any valid value for the input of the second layer and use it to repeat the above extraction process. But different from the first layer, we do not need to probe critical points for all kernels of the second layer, since we just need one critical for extracting model parameters of a kernel. Therefore, the probing process of the second layer just needs to probe critical points for the remaining kernels, which means the probe may terminate early.

In the worst case, we cannot reuse anything and have to launch a new probe in each layer for all kernels. The average number of queries required for extracting an L -layer CNN can be approximated as:

$$\begin{aligned} & \sum_{i=1}^L \left(N_p^{(i)} \cdot 2 + N_c^{(i)} \cdot 2 \frac{r_i^2}{k^2} + 2c_i c_{i-1} k^2 \right) \\ & \approx \sum_{i=1}^L \left(2c_i(L-i) \log_2 \frac{\lambda}{c_i(L-i)} + \frac{2}{3} c_i(L-i)^3 + 2c_i c_{i-1} k^2 \right) \end{aligned} \quad (11)$$

3) *Comparison of Required Resources:* As analyzed and summarized in Equ. (8) and Equ. (11), there are two major components in the number of queries for both attack methods. The first component shows the effort in searching

Table I
NUMBER OF QUERIES TO ATTACK SOME COMMON CNNs

Architecture	Parameters	Number of Queries	
		Adaptive attacking PIPO	Prediction API attack (wst, exp)
AlexNet	37.5M	$2^{23.0}$	$2^{23.0}, 2^{23.0}$
MiniONN	161K	$2^{18.7}$	$2^{19.1}, 2^{18.9}$
VGG-16	14.7M	$2^{25.3}$	$2^{26.4}, 2^{26.0}$
VGG-19	20M	$2^{25.8}$	$2^{26.4}, 2^{26.0}$
ResNet-18	4.7M	$2^{22.6}$	$2^{23.5}, 2^{23.3}$
ResNet-32	9.8M	$2^{24.2}$	$2^{24.9}, 2^{24.6}$

for the critical points or turning points of ReLU functions, parameterized with $\log_2 \lambda$. For each layer i , the adaptive attack takes $c_i \log_2 \lambda$ queries, while the prediction API attack takes $c_i(L-i) \log_2 \frac{\lambda}{c_i(L-i)}$ queries on average. The prediction API attack is more efficient in this component. The second component relates to the effort in extracting model parameters. The adaptive attack takes $\frac{2(L-i)^2}{c_i}$ queries on average for each parameter in layer i , while the prediction API attack takes $2 + \frac{2(L-i)^3}{3c_{i-1}k^2}$ queries on average. The adaptive attack is more efficient in this component, especially for deeper networks.

We compare the number of queries required by the two methods using several common CNN architectures in Table I. To make a fair comparison and demonstrate the difference between different CNN architectures, we focus on the convolution part of these networks and only keep one fully connected layer at the end. The inputs are color images with 3 channels. We search in range $[-500, 500]$ with granularity 10^{-9} i.e., $\lambda = 10^{12}$. The result shows that the number of queries required by the adaptive attack against PIPO is close to the prediction API attack. And it is at least half of the prediction API attack on AlexNet, MiniONN, VGG, and ResNet.

VI. EVALUATION

In this section, we evaluate our system PIPO in terms of its performance and accuracy. We first present the experimental setup in Section VI-A. Then, we evaluate the performance of our system in terms of the inference latency and the communication cost in Section VI-B. After that, we evaluate the performance of activation layers in Section VI-C and pooling layers in Section VI-D.

A. Experimental Setup

All our experiments were carried out on AWS C5.2XLARGE instances possessing an Intel Xeon 8000 series machine CPU at 3.0 GHz with 16 GB of RAM. We conduct experiments on the following network architectures and corresponding datasets.

First, we adopt the MiniONN [16] network. It contains 7 convolution layers, 7 activation layers, 2 pooling layers, and 1 fully-connected layer. We evaluate it on the CIFAR-10 dataset [25]. It consists of 60,000 RGB images of size 32x32. Images are classified into 10 classes. The training set and test sets have 50,000 and 10,000 images respectively.

Second, we adopt a more complex ResNet-32 [26] neural network. It contains 33 convolution layers, 33 activation layers, 16 shortcut layers, and 1 fully-connected layer. It does not have pooling layers. It summarizes the image features using convolutions with stride 2x2, which reduces the height and width of the image by half. We evaluate the network with the CIFAR-100 dataset [25]. Its dataset size and image size are the same as the CIFAR-10 dataset, but the images are classified into 100 classes instead of 10.

We compare our system with Gazelle [10], Delphi [1], CryptoNAS [13], DeepReDuce [14], Circa [15], LLAMA [17], and Cheetah [27]. Gazelle employs homomorphic encryption to process linear layers and uses garbled circuits to process the non-linear ReLU activations. Delphi processes linear layers in plain text. For non-linear operations, it performs ReLU activations using garbled circuits as Gazelle, while it also replaces some ReLU operations with quadratic approximations using the Beaver’s multiplication procedure [28]. CryptoNAS and DeepReDuce employ the network architecture search (NAS) technique to find a network architecture that balances the GC-based ReLU and its approximations. Circa optimizes the garbled circuits for the ReLU operations. LLAMA performs neural network inference through the function secret sharing (FSS) techniques. Cheetah designs a highly optimized HE-based protocol.

We implement our system in Python. The code is open source at GitHub.¹ We use the PyTorch library for neural network processing, Pyfhel, and the Microsoft SEAL library for homomorphic encryption.

B. End-to-End Performance

In Table II, we compare our system PIPO with existing systems. We compare the systems in terms of running time, communication cost, and prediction accuracy. The result shows that PIPO is about 20x faster than Delphi on the MiniONN network and 22x faster on ResNet-32. In terms of the communication cost, PIPO transfers 4x and 26x less data than Delphi on MiniONN and ResNet-32 respectively. In addition, our system improves the prediction accuracy over the all-ReLU version of Delphi by about 4.0 % and 8.6 % on MiniONN and ResNet-32 respectively.

PIPO achieves better accuracy while running faster than state-of-the-art systems because of the following two reasons. First, PIPO performs the activation with the exact ReLU function. All other systems approximate some ReLU operations. The approximation usually introduces errors, which leads to lower accuracy. Second, PIPO supports the max pooling operation with efficient plaintext operations. Many existing systems, such as Gazelle, Delphi, CryptoNAS, approximate max pooling with average pooling and retrain the network for this modification. However, from the perspective of CNN design, the max pooling is used to select the most significant feature in a region and ignore the rest. Since PIPO employs plaintext operations (multiplication and shuffle), it is much

Table II
PERFORMANCE COMPARISON ON INFERENCE TIME, COMMUNICATION COST, AND ACCURACY

Network	System	Time(s)	Comm.(MB)	Accuracy
MiniONN	PIPO	0.02	1.8	90.3%
	LLAMA	0.52	8.2	-
	Delphi	0.41	7	82.7%
	Delphi (ReLU)	9.0	164	86.3%
	Gazelle	48.8	279	86.3%
ResNet-32	PIPO	0.06	3.0	76.3%
	DeepReDuce-28.7k	0.74	-	68.7%
	DeepReDuce-49.2k	1.19	-	69.5%
	CryptoNAS-50k*	1.67	-	63.6%
	Circa	2.47	-	66.4%
	Delphi	4.70	78	65.7%
	CryptoNAS-334k*	7.51	-	75.5%
	Cheetah	15.95	110	67.8%
	Delphi (ReLU)	17.7	280	67.8%
	Gazelle	82.2	574	67.8%

*: The CNN used by CryptoNAS is not exactly the ResNet-32 model.

more efficient than systems based on garbled circuits (such as Gazelle, Delphi, Circa) or function secret sharing (LLAMA).

PIPO’s performance and accuracy advantages are more obvious on deeper CNN architectures. Compared with the MiniONN architecture, the ResNet-23 architecture is 4 times deeper and correspondingly involves more ReLU operations. The error introduced by the approximation of ReLU and max pooling gets accumulated and becomes more significant in deeper networks. By executing exact ReLU and max pooling functions, PIPO reduces the noise and gets better accuracy.

C. Performance of Activation Layers

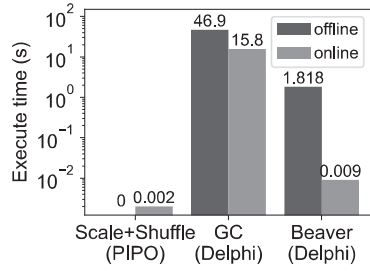
ReLU activation is the most common function in many neural networks. The GC-based ReLU function is usually the most time-consuming part of many privacy-preserving inference systems. Some approximated ReLU activation functions, such as the square function [13] or polynomial functions [1], [12], [15], usually accelerate the activation at the cost of accuracy loss.

We evaluate the performance of the ReLU activation layer in PIPO. Focusing on the activation, we use ResNet-32 as the benchmark model in these experiments because there is no pooling layer. We measure the time cost and communication cost of activation layers in the model. We compare the performance of ReLU in PIPO with the ReLU function based on garbled circuits and quadratic approximation with Beaver’s multiplication protocol using Delphi’s implementation.

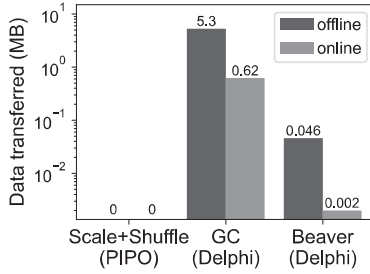
Fig. 6(a) compares time cost of ReLU in PIPO and the two methods in Delphi. As shown in the figure, ReLU in PIPO does not require offline preparation. For the online inference phase, PIPO’s ReLU is about 600x faster than the implementation based on GC. And PIPO’s ReLU is 4.7x faster than the approximation implementation based on the Beaver’s multiplication protocol while being more accurate.

Fig. 6(b) shows the communication cost. Since PIPO is able to perform the ReLU computation locally on the client, the ReLU operation itself incurs zero data transfer. In contrast, the

¹PIPO Code: <https://github.com/yxtj/PIPO>



(a) Running time



(b) Communication cost

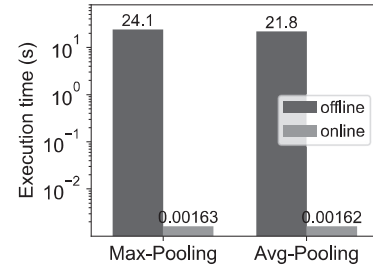
Fig. 6. Performance of Activation Layers (on ResNet-32)

GC version exchanges a ciphertext for each bit of each number between the client and the server. The approximated version using Beaver's multiplication just exchanges a few scalars for each number.

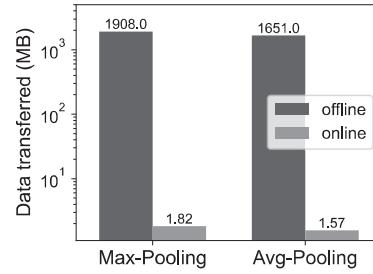
As shown in Table II, the accuracy of ResNet-32 in PIPO is higher than that in Delphi. The results show that PIPO is 8.6% and 10.7% more accurate than Delphi's all GC implementation and GC with approximation implementation. It is because that PIPO implements the exact ReLU activation and max-pooling operations without approximation. Since there is only one pooling layer in ResNet-32, most of the accuracy gain comes from the ReLU implementation. The garbled circuit implementation should reach the same accuracy as our scaling-based method. However, it requires the input data of the garbled circuit to have the same amount of precision bits as our input. However, more bits in garbled circuits take longer processing time. In practice, it is necessary to balance the precision and latency. For example, Delphi represents the input with 15-bit fixed-point numbers. While PIPO uses 32-bit for each number.

D. Impact and Performance of Max-Pooling Layers

Pooling operation plays an important role in many CNNs. PIPO supports efficient max pooling operations. Different from most privacy-preserving inference systems [1], [8]–[10], which approximate the max pooling operation with average pooling operation, PIPO performs the exact max pooling operation in neural networks. We evaluate the efficiency of max pooling affects the performance with MiniONN. Shown in Table II, approximating with average pooling, the best accuracy we can get is 86.3%, while the accuracy using max pooling is 90.3%.



(a) Running time



(b) Communication cost

Fig. 7. Performance of Max and Average Pooling (on MiniONN)

We compare running time and communication cost in Fig. 7. At the cost of better accuracy, the max pooling version takes about 12% longer time and 15% more data in the offline processing phase. In the online phase, even the max pooling version still transfers about 15% more data, the total volume is just 1.82 MB. As a result, there is no noticeable difference between the time cost by the two versions.

VII. RELATED WORKS

This section introduces some related works about privacy-preserving neural network inference, in terms of protocol efficiency and protocol security.

A. Privacy-Preserving Inference

Homomorphic encryption is a form of encryption that allows mathematical operations on encrypted data to be carried out, producing an encrypted result that is equivalent to the result of the operations on the original unencrypted data [6], [8]. Based on that, some systems [9], [10], [29] are proposed to perform neural network inference on encrypted data using HE schemes. They submit encrypted input data to the server for computation and send back the ciphertext of results to the client. However, the huge computation workload of HE lowers the throughput of these systems and makes them far away from practice [8]–[10].

Delphi [1] accelerates the inference by pushing the time-consuming HE operations to an offline phase. It applies the additive secret sharing technique [30] to separate the computation of a linear function into two parts. It computes one part with HE in the offline phase and computes the other part with

a lightweight two-party computation protocol in the online phase. It reduces computation and communication costs.

Many HE schemes [7], [31] do not support non-linear operations such as ReLU and max pooling. So many works approximate non-linear operations with HE-supported operations [1], [9], [10], [12], [32]. For example, CryptoNet [9] replaces the max-pooling operation with an average-pooling operation and approximates ReLU with a square function. Many systems follow the practice [1], [16]. However, it unavoidably reduces the accuracy of the neural network. To mitigate the accuracy loss, efficient and accurate approximation methods for HE are proposed [12], [32]. They adopt higher order polynomial functions to approximate the non-linear operations and study the efficient way of composing the polynomial functions.

In addition to HE, some researchers adopt secure multi-party computation techniques to perform non-linear operations in neural network inference. Systems such as Gazelle and Delphi [1], [10], [11], [15], [16] use Garbled circuit (GC) [19] to implement non-linear functions. They represent non-linear computations with boolean circuits. During the inference, each bit of the non-linear function's input and output data is encrypted individually and transferred via oblivious transfer [33]. As a generic tool, GC is also used to implement linear operations, as in XONN [34].

Recently, some works propose to use the function secret sharing (FSS) technique [20], [21] for privacy-preserving neural network inferences [17], [18], [35]. FSS separates the computation of a function into two parts and makes the summation of their results equal to the result of the original function. Compared with GC, it is more efficient since it does not involve encryption/decryption operations for online inference.

The approximation idea is also adopted in SMPC-based inference systems. They approximate the ReLU function with polynomial functions and apply Beaver's multiplication protocol (BMP) [28] to securely compute the approximated functions [1], [10], [16]. On top of that, some researchers propose to approximate a selected set of ReLU operations instead of all ReLU operations [1], [13], [14] to balance performance and accuracy.

B. Security of Model Parameters

Since the prediction results unavoidably contain information about the model parameters, a curious adversary can use this information to learn the model parameters or the training data through the prediction API [36], [37]. In a naive approach, an adversary can collect many prediction results and use them to train a model that approximates the server's model. Following this training-based attack idea, some methods are proposed to augment the data collection [38], [39].

Another branch of the attacking methods is to reconstruct a network by analyzing the prediction results [22]–[24], [40], [41]. For ReLU-based neural networks, Hanin *et al.* [40] consider the boundaries between linear regions as bent hyperplanes. Recent works [22]–[24], [41] extract the model parameters at critical points on the bent hyperplanes. These methods

focus on fully connected networks for better generality. In this paper, to understand the security of our protocol, our CNN-targeted prediction API attack utilizes convolution properties to greatly reduce the requirement of the number of queries.

Currently, there is no perfect defense against prediction API attacks. We can slow down the attack by rate limiting, query auditing [41], [42], and adding noise to the prediction results [43], [44].

VIII. CONCLUSION

In this paper, we propose PIPO for privacy-preserving convolution neural network inference. It divides an inference into an offline preprocessing phase and an online inference phase and only applies plaintext operations (multiplication and shuffle) in the online phase. By avoiding expensive cryptographic operations, such as homomorphic encryption and garbled circuits, in the online inference phase, PIPO accelerates the privacy-preserving inference by 20-80 times over state-of-the-art systems. In addition, due to PIPO's efficient implementation for ReLU and max pooling, we do not need to approximate these operations as existing systems in exchange for lower latency. Therefore, PIPO achieves both high prediction accuracy and low inference latency. For security, we argue the semi-honest security of PIPO for user data with a simulation-based proof. Since an attacker can always extract model parameters using seldom the inputs and outputs, we show that the number of queries for stealing model parameters in PIPO is at least half of the state-of-the-art prediction API attack method for CNN.

IX. ACKNOWLEDGMENT

This work was supported in part by National Science Foundation Grants CNS-1918187, CNS-1908536.

REFERENCES

- [1] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 2505–2522.
- [2] T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, "Bag of tricks for image classification with convolutional neural networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 558–567.
- [3] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *Advances in neural information processing systems*, vol. 28, pp. 91–99, 2015.
- [4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [5] T. Zhou and L. Gao, "Distributed framework for accelerating training of deep learning models through prioritization," in *2021 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 2021, pp. 201–209.
- [6] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proceedings of the forty-first annual ACM symposium on Theory of computing*, 2009, pp. 169–178.
- [7] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2017, pp. 409–437.
- [8] I. Chillotti, M. Joye, and P. Paillier, "Programmable bootstrapping enables efficient homomorphic inference of deep neural networks," *IACR Cryptol. ePrint Arch.*, vol. 2021, p. 91, 2021.

- [9] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *International conference on machine learning*. PMLR, 2016, pp. 201–210.
- [10] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "Gazelle: A low latency framework for secure neural network inference," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 1651–1669.
- [11] B. Reagen, W.-S. Choi, Y. Ko, V. T. Lee, H.-H. S. Lee, G.-Y. Wei, and D. Brooks, "Cheetah: Optimizing and accelerating homomorphic encryption for private inference," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 26–39.
- [12] J. Lee, E. Lee, J.-W. Lee, Y. Kim, Y.-S. Kim, and J.-S. No, "Precise approximation of convolutional neural networks for homomorphically encrypted data," *arXiv preprint arXiv:2105.10879*, 2021.
- [13] Z. Ghodsi, A. K. Veldanda, B. Reagen, and S. Garg, "Cryptonas: Private inference on a relu budget," *Advances in Neural Information Processing Systems*, vol. 33, pp. 16961–16971, 2020.
- [14] N. K. Jha, Z. Ghodsi, S. Garg, and B. Reagen, "Deepreduce: Relu reduction for fast private inference," in *International Conference on Machine Learning*. PMLR, 2021, pp. 4839–4849.
- [15] Z. Ghodsi, N. K. Jha, B. Reagen, and S. Garg, "Circa: Stochastic relus for private deep learning," *Advances in Neural Information Processing Systems*, vol. 34, pp. 2241–2252, 2021.
- [16] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minionn transformations," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 619–631.
- [17] K. Gupta, D. Kumaraswamy, N. Chandran, and D. Gupta, "Llama: A low latency math library for secure inference," *Proceedings on Privacy Enhancing Technologies*, no. 4, pp. 274–294, 2022.
- [18] M. Hao, H. Li, H. Chen, P. Xing, and T. Zhang, "Fastsecnet: An efficient cryptographic framework for private neural network inference," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 2569–2582, 2023.
- [19] A. C.-C. Yao, "How to generate and exchange secrets," in *27th Annual Symposium on Foundations of Computer Science (sfcs 1986)*. IEEE, 1986, pp. 162–167.
- [20] E. Boyle, N. Gilboa, and Y. Ishai, "Function secret sharing," in *Advances in Cryptology-EUROCRYPT 2015: 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II*. Springer, 2015, pp. 337–367.
- [21] —, "Function secret sharing: Improvements and extensions," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 1292–1303.
- [22] D. Rolnick and K. Kording, "Reverse-engineering deep relu networks," in *International Conference on Machine Learning*. PMLR, 2020, pp. 8178–8187.
- [23] N. Carlini, M. Jagielski, and I. Mironov, "Cryptanalytic extraction of neural network models," in *Annual International Cryptology Conference*. Springer, 2020, pp. 189–218.
- [24] I. A. Canales-Martínez, J. Chavez-Saab, A. Hambitzer, F. Rodríguez-Henríquez, N. Satpute, and A. Shamir, "Polynomial time cryptanalytic extraction of neural network models," *Cryptology ePrint Archive*, 2023.
- [25] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [27] Z. Huang, W.-j. Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure {Two-Party} deep neural network inference," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 809–826.
- [28] B. Donald, "Precomputing oblivious transfer," in *CRYPTO'95*, 1995.
- [29] Q. Lou, B. Feng, G. Charles Fox, and L. Jiang, "Glyph: Fast and accurately training deep neural networks on encrypted data," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9193–9202, 2020.
- [30] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [31] C. Gentry, A. Sahai, and B. Waters, "Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based," in *Annual Cryptology Conference*. Springer, 2013, pp. 75–92.
- [32] E. Lee, J.-W. Lee, Y.-S. Kim, and J.-S. No, "Minimax approximation of sign function by composite polynomial for homomorphic comparison," *IEEE Transactions on Dependable and Secure Computing*, 2021.
- [33] M. O. Rabin, "How to exchange secrets with oblivious transfer," *Cryptology ePrint Archive*, 1981.
- [34] M. S. Riazzi, M. Samragh, H. Chen, K. Laine, K. E. Lauter, and F. Koushanfar, "Xonn: Xnor-based oblivious deep neural network inference," in *USENIX Security Symposium*, 2019, pp. 1501–1518.
- [35] T. Ryffel, P. Tholoniati, D. Pointcheval, and F. Bach, "Ariann: Low-interaction privacy-preserving deep learning via function secret sharing," *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 1, pp. 291–316, 2022.
- [36] G. Ateniese, L. V. Mancini, A. Spognardi, A. Villani, D. Vitali, and G. Felici, "Hacking smart machines with smarter ones: How to extract meaningful data from machine learning classifiers," *International Journal of Security and Networks*, vol. 10, no. 3, pp. 137–150, 2015.
- [37] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE symposium on security and privacy (SP)*. IEEE, 2017, pp. 3–18.
- [38] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, 2017, pp. 506–519.
- [39] S. Goel and A. R. Klivans, "Learning neural networks with two nonlinear layers in polynomial time," in *Conference on Learning Theory*. PMLR, 2019, pp. 1470–1499.
- [40] B. Hanin and D. Rolnick, "Deep relu networks have surprisingly few activation patterns," *Advances in neural information processing systems*, vol. 32, 2019.
- [41] M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, and N. Papernot, "High accuracy and high fidelity extraction of neural networks," in *Proceedings of the 29th USENIX Conference on Security Symposium*, 2020, pp. 1345–1362.
- [42] H. Li, S. Shan, E. Wenger, J. Zhang, H. Zheng, and B. Y. Zhao, "Blacklight: Scalable defense for neural networks against query-based black-box attacks," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 2117–2134.
- [43] T. Orekondy, B. Schiele, and M. Fritz, "Prediction poisoning: towards defenses against dnn model stealing attacks," *arXiv preprint arXiv:1906.10908*, 2019.
- [44] J. Zhang, S. Peng, Y. Gao, Z. Zhang, and Q. Hong, "Apmsa: Adversarial perturbation against model stealing attacks," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 1667–1679, 2023.